CI/CD PIPELINE WITH AWS CODEPIPELINE, CODEBUILD AND CODEDEPLOY

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT

Under the guidance of

Mrs. Bethala Sumana, AWS Mentor

Mr. Juttuka Lovababu, AWS Mentor

Submitted by

MORTHA TENNY SAMUEL SWARAJ, 258865100036, ADIKAVI NANNAYA UNIVERSITY, RJY, (Batch-1)

DASARI PAVANI SURYAKUMARI, 258865100011, ADIKAVI NANNAYA UNIVERSITY, RJY, (Batch-1)

YALAMANCHILI VENI VANDHANA, 258865100055, ADIKAVI NANNAYA UNIVERSITY, RJY, (Batch-1)

MEDISETTI SANJANA, 258865100033, ADIKAVI NANNAYA UNIVERSITY, RAJAMUNDRY, (Batch-1)

UNDURTHI KOUSALYA, 258865100052, ADIKAVI NANNAYA UNIVERSITY, RAJAMUNDRY, (Batch-1)

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who have contributed to the successful completion of our summer internship project. This opportunity has been an enriching and transformative experience for us, and we are truly thankful for the support, guidance, and encouragement we have received along the way.

First and foremost, we extend our sincere regards and gratitude to our supervisors and mentors, Mrs. Bethala Sumana and Mr. Juttuka Lovababu, for providing us with valuable insights, constant guidance, and unwavering support throughout the duration of this internship. Their expertise and encouragement have been instrumental in shaping the direction and implementation of this project.

We would also like to thank the faculty and administration of Adikavi Nannaya University, Rajahmundry, for supporting us during this internship program and providing us with the necessary academic foundation to undertake this work.

Finally, we are grateful to AWS and the organizers of this Internship Program for providing us with a platform to gain hands-on experience in cloud infrastructure and DevOps practices.

Sincerely,

Project Team Members:

MORTHA TENNY SAMUEL SWARAJ

DASARI PAVANI SURYAKUMARI

YALAMANCHILI VENI VANDHANA

MEDISETTI SANJANA

UNDURTHI KOUSALYA

ABSTRACT

In modern software engineering, manual application deployment processes pose significant bottlenecks. Activities such as manually transferring build artifacts to web servers, handling version conflicts, restarting daemon processes, and verifying system uptime are inherently slow and highly prone to human error. To address this challenge, this project implements a fully automated **Continuous Integration and Continuous Deployment (CI/CD) pipeline on Amazon Web Services (AWS)** to orchestrate the lifecycle of a student portfolio website.

The system architecture utilizes a simple multi-tier configuration designed for high availability and automated scale. The web application frontend is developed using standard **web technologies** (HTML5, CSS3, and JavaScript) and **version-controlled on GitHub**. When a developer pushes code changes to the remote repository, AWS CodePipeline detects the event and initiates the workflow. AWS CodeBuild retrieves the updated source code, verifies that required project configurations exist, and compiles the build packages, saving the output artifacts securely in an Amazon S3 bucket. AWS CodeDeploy then fetches these packages and deploys them to Amazon EC2 instances. The EC2 instances are structured within an Auto Scaling Group (ASG) to ensure elasticity, and are placed behind an **Application Load Balancer (ALB)** to balance incoming HTTP requests across availability zones.

Through this project, we demonstrate a complete production-ready pipeline that automates the deployment lifecycle. We also explore troubleshooting strategies for common deployment hurdles, such as CodeDeploy agent configurations and local directory clearance conflicts. The result is a robust, self-healing, and zero-downtime deployment framework that highlights the power of AWS DevOps tools in simplifying software delivery.

TABLE OF CONTENTS

1. INTRODUCTION

In the fast-paced landscape of cloud computing and software delivery, organization agility is highly dependent on how quickly and safely code modifications can be deployed to production servers. Traditional web deployment methodologies involve manual steps carried out by system administrators or developers. These include transferring source files via FTP or SFTP, manually replacing target directory folders, stopping and restarting web server services (e.g., Apache HTTP Server or Nginx), and executing post-deployment validation checks. This approach creates several difficulties:

- Manual interventions introduce human errors, such as uploading incorrect versions of files or omitting configurations.
- Servers can experience unexpected downtime during the replacement of active application binaries or configuration updates.
- Tracking historical versions and rolling back to a previous stable state is difficult and time-consuming in emergency failure scenarios.
- Scaling updates across a cluster of multiple virtual servers requires repeating manual steps on each machine, which is highly inefficient.

DevOps methodologies solve these problems by introducing automation to the integration and deployment cycles. This project implements a fully automated DevOps pipeline using AWS cloud services. By defining infrastructure and build rules as code (declarative YAML files), we eliminate manual file uploads and manual server configurations. This document outlines the design, architecture, configuration scripts, and results of setting up a Continuous Integration and Continuous Deployment (CI/CD) pipeline for a responsive Student Portfolio Website.

2. METHODOLOGY

The execution of this project followed a structured, iterative cloud-development lifecycle designed to align with best practices in AWS architecture. The methodology consists of the following key phases:

2.1. Requirements Analysis & Code Creation

First, we defined the requirements for our application and hosting infrastructure. The website needs to be a standard, responsive Student Portfolio Website. It consists of index.html (structure), style.css (styling), and script.js (interactivity). Version control was managed via GitHub to enable distributed code development and trigger-based deployment actions.

2.2. Cloud Infrastructure Design

To host the application reliably, we designed a high-availability infrastructure in AWS. We configured virtual servers (EC2 instances) within an Auto Scaling Group (ASG) to dynamically maintain instance availability. An Application Load Balancer (ALB) was positioned at the front of the compute layer to provide users with a single, unified entry point (DNS URL) and to distribute load across EC2 instances in multiple Availability Zones.

2.3. Continuous Integration Setup

We set up AWS CodeBuild to automate the packaging and verification of code files. CodeBuild is configured via a buildspec.yml file. It automatically compiles changes, structures the artifacts, and stores them in a secure Amazon S3 bucket.

2.4. Continuous Deployment Setup

AWS CodeDeploy was chosen as the agent-based deployment orchestrator. CodeDeploy is governed by an appspec.yml file and helper shell scripts. It automates copying files into the Apache Document Root (/var/www/html), executing dependency scripts, and restarting the service without manual shell access.

2.5. Pipeline Orchestration

AWS CodePipeline connects the separate services into a single, unified workflow. The pipeline automates the transition from Source (GitHub push) -> Build (AWS CodeBuild) -> Deploy (AWS CodeDeploy to Auto Scaling Group).

2.6. Testing & Troubleshooting

We validated the system under active code modifications to check that the pipeline successfully executed each stage. During this process, we identified and resolved real-world issues such as missing system agent daemons and file collision errors on the target servers.

3. TECHNOLOGY STACK

The technology stack chosen for this project consists of modern, industry-standard web frontend technologies, version control platforms, and AWS DevOps and infrastructure services:

3.1. Front-end Technologies

- HTML5: Used to establish the structure, content, layout, and document semantics of the Student Portfolio Website.
- CSS3: Applied to define custom colors, responsive grid layouts, media queries for mobile scaling, and hover animations.

- JavaScript (ES6): Used for client-side interactivity, smooth page scrolling, and dynamic elements on the portfolio.

3.2. Version Control & Source Code Management

- Git & GitHub: Managed code commits, branch histories, and provided integration with AWS via the GitHub Webhook connection.

3.3. AWS Infrastructure Services

- Amazon EC2 (Elastic Compute Cloud): Provides resizable virtual compute capacity (t3. micro instances running Amazon Linux 2023) to host the Apache web server.
- Auto Scaling Group (ASG): Automatically launches or terminates EC2 instances to maintain a desired capacity of 2 instances, scaling between a minimum of 2 and maximum of 4 based on traffic patterns.
- Application Load Balancer (ALB): Operates at the application layer (Layer 7) to distribute incoming HTTP traffic to healthy targets within the Target Group across eu-north-1a and eu-north-1b subnets.
- Amazon S3 (Simple Storage Service): Serves as the central repository for intermediate build artifacts produced by CodeBuild before they are deployed.

3.4. AWS DevOps Services

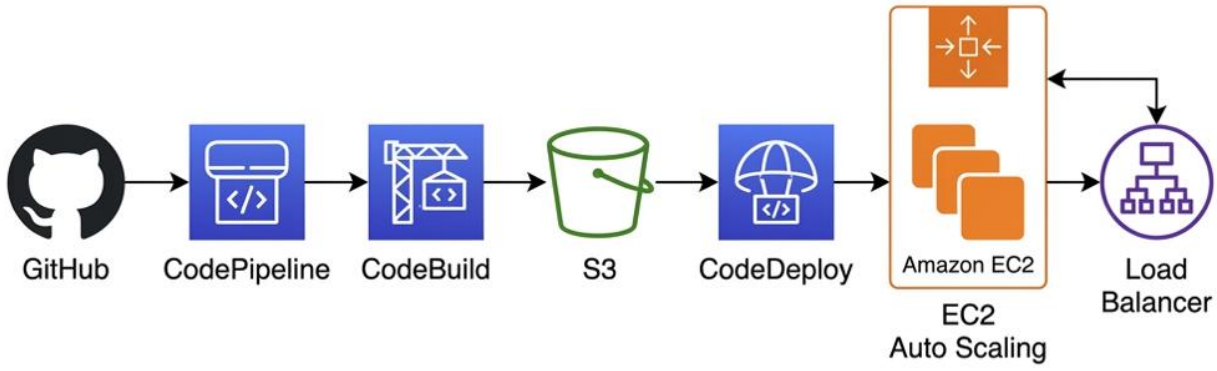
- AWS CodePipeline: Orchestrates the pipeline stages (Source -> Build -> Deploy) and manages execution state.
- AWS CodeBuild: A fully managed build service that packages the static site files and confirms project file presence.
- AWS CodeDeploy: Deploys code to EC2 instances using the appspec.yml lifecycle configurations.

3.5. Security & Management

- AWS IAM (Identity and Access Management): Establishes fine-grained security policies, allowing CodeBuild to access S3 and granting CodeDeploy permissions to interact with EC2 APIs under the principle of least privilege.
- Amazon CloudWatch: Aggregates logs from CodeBuild and EC2 systems for real-time tracking and troubleshooting.

4. SYSTEM DESIGN / ARCHITECTURE

The system architecture represents an event-driven DevOps pipeline paired with a highly available web server cluster. The diagram below illustrates the logical flow of code modifications from the developer's workstation to the end-users accessing the live website:



The flow of operations is structured as follows:

- **Source Phase:** 1. Code Commits: The developer writes code locally and pushes the modifications to the GitHub repository main branch.
- **Pipeline Orchestration:** 2. Pipeline Trigger: AWS CodePipeline detects the new commit via GitHub integration and pulls the latest source code archive.
- **Build Phase:** 3. Code Compilation: CodeBuild compiles the source, runs build scripts specified in buildspec.yml, packages files, and uploads the deployable artifact to Amazon S3.
- **Deployment Phase:** 4. Deployment & Launch: CodePipeline triggers CodeDeploy, which retrieves the build artifact from S3. The CodeDeploy agent on each EC2 instance parses appspec.yml, runs the setup shell scripts, copies the web files to the Apache server root, and restarts the web service.
- **Load Balancing Phase:** 5. Load Balancing & Traffic: The Application Load Balancer continuously monitors the health of the EC2 instances. Users access the load balancer's DNS name, and the ALB routes traffic to healthy instances in the target group, ensuring continuous availability.

5. IMPLEMENTATION DETAILS

This section describes the configuration and integration details of the files, scripts, and AWS services used to implement the automated pipeline.

5.1. GitHub Source Control Structure

The code files of the Student Portfolio Website were structured to contain both the website assets and the configuration files needed by the AWS build and deploy services. The project structure is shown below:

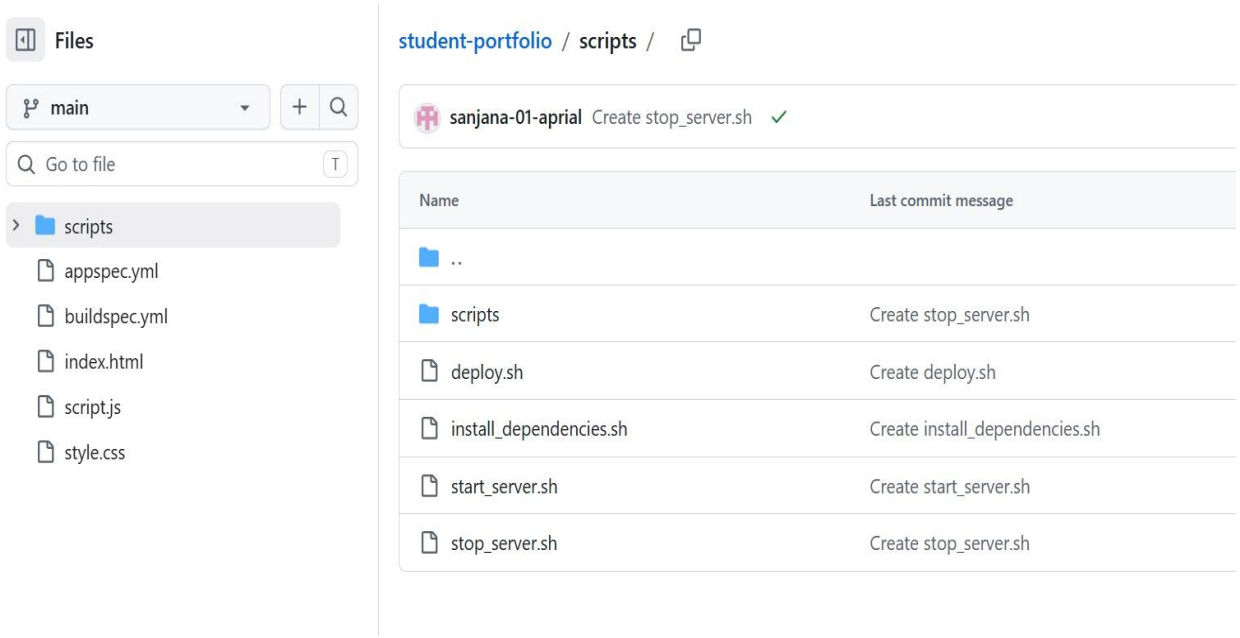


Figure 1: GitHub Repository File Structure showing web assets and AWS configuration files

5.2. Continuous Integration Configurations (buildspec.yml)

The buildspec.yml file defines the commands that AWS CodeBuild executes. Because our portfolio website is static, CodeBuild checks that all source files are present and archives them as a zip artifact. Below is the buildspec.yml implementation:

```
version: 0.2

phases:
  install:
    runtime-versions:
      nodejs: latest
    commands:
      - echo 'Installing dependencies...'
  pre_build:
    commands:
      - echo 'Verifying structure...'
      - ls -la
  build:
    commands:
      - echo 'Packaging website files...'
  post_build:
    commands:
      - echo 'Build completed successfully.'
artifacts:
  files:
    - '**/*'
```

5.3. Continuous Deployment Configurations (appspec.yml)

The appspec.yml file is used by AWS CodeDeploy to manage the deployment lifecycle on EC2. It specifies which files should be copied and lists the shell scripts to execute at different phases of the deployment:

```
version: 0.0
os: linux
files:
  - source: /
    destination: /var/www/html
hooks:
  BeforeInstall:
    - location: scripts/install_dependencies.sh
      timeout: 300
      runas: root
    - location: scripts/deploy.sh
      timeout: 300
      runas: root
  ApplicationStop:
    - location: scripts/stop_server.sh
      timeout: 300
      runas: root
  ApplicationStart:
    - location: scripts/start_server.sh
      timeout: 300
      runas: root
```

5.4. Deployment Helper Shell Scripts

To support the CodeDeploy lifecycle hooks, several shell scripts were created in the scripts directory:

scripts/install_dependencies.sh

This script runs during the BeforeInstall hook to install the Apache HTTP server:

```
#!/bin/bash
# Install Apache web server
yum update -y
yum install -y httpd
```

scripts/deploy.sh

This script runs before copying files to clear out existing web content (resolving directory conflicts):

```
#!/bin/bash
# Remove index.html if it exists to prevent deployment failure
if [ -f /var/www/html/index.html ]; then
    rm -f /var/www/html/index.html
fi
rm -rf /var/www/html/*
```

scripts/stop_server.sh

This script stops the active web server prior to file updates:

```
#!/bin/bash
# Stop Apache server to prepare for code refresh
systemctl stop httpd || true
```

scripts/start_server.sh

This script starts the web server and enables it to boot automatically after file transfer completes:

```
#!/bin/bash
# Start Apache server and enable on startup
systemctl start httpd
systemctl enable httpd
```

Once all configurations were pushed to the GitHub repository, we verified the pipeline execution. This section outlines the setup results of the AWS infrastructure and the troubleshooting solutions applied.

6.1. AWS Auto Scaling Group Verification

The Auto Scaling Group successfully launched EC2 instances to maintain the configured capacity. The instances were created using the launch template, which is configured with an AMI running Amazon Linux:

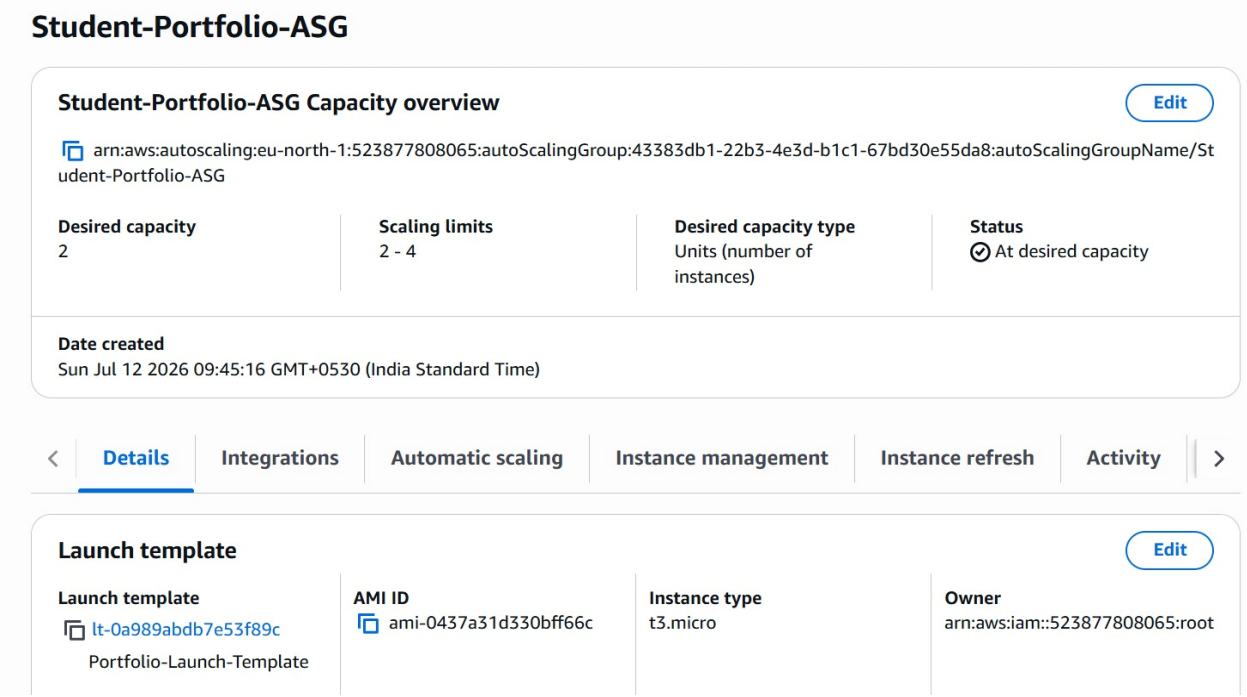


Figure 2: AWS Auto Scaling Group configuration showing a desired capacity of 2 instances

6.2. Application Load Balancer Configuration

The Application Load Balancer was verified to be in an active state. The public DNS name was generated, providing a single point of entry for user requests:

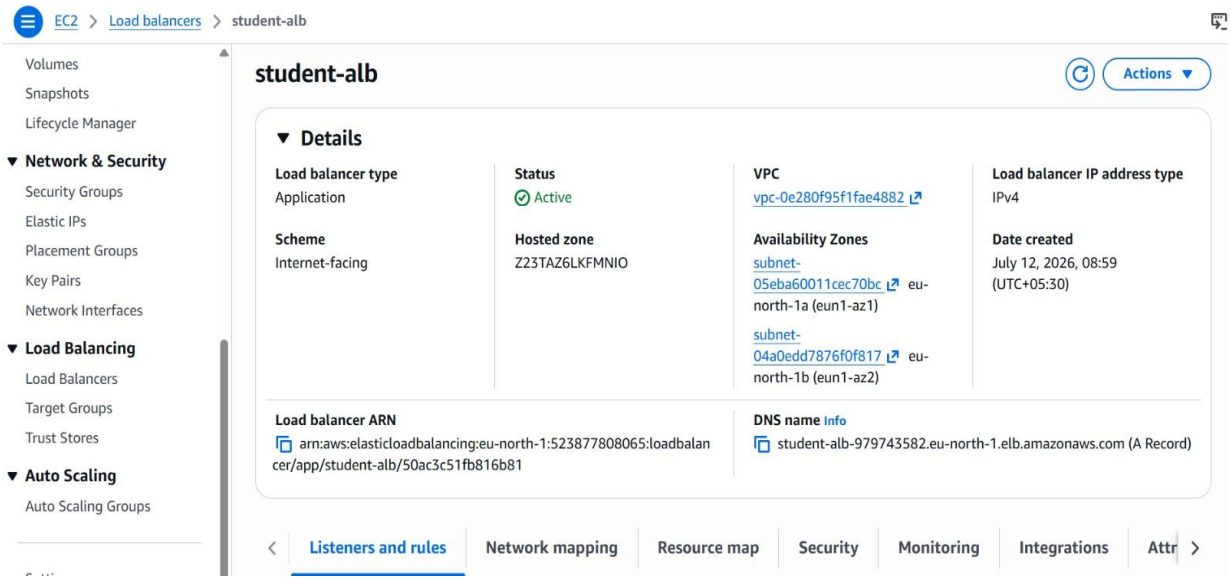


Figure 3: AWS Application Load Balancer details showing the active state and DNS URL

6.3. Target Group Health Status

The Application Load Balancer's target group successfully registered two EC2 instances. Both targets were verified as healthy, confirming that the web servers are running and accessible on port 80:

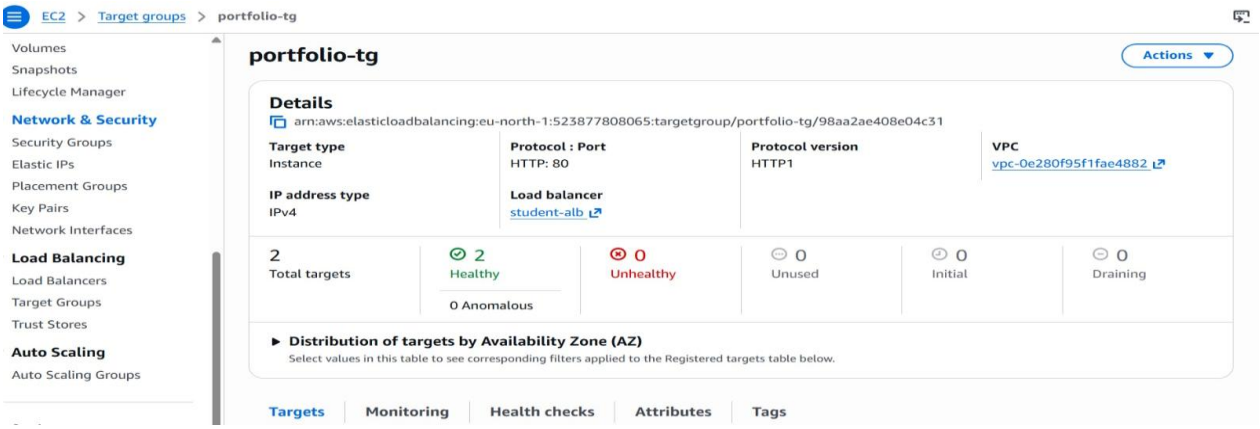


Figure 4: Target Group details confirming 2 healthy targets registered to the load balancer

6.4. CodePipeline Stage Execution

We verified the pipeline under an active commit. CodePipeline successfully detected the code change from GitHub, built the package using CodeBuild, and deployed the updates to the Auto Scaling Group instances via CodeDeploy:

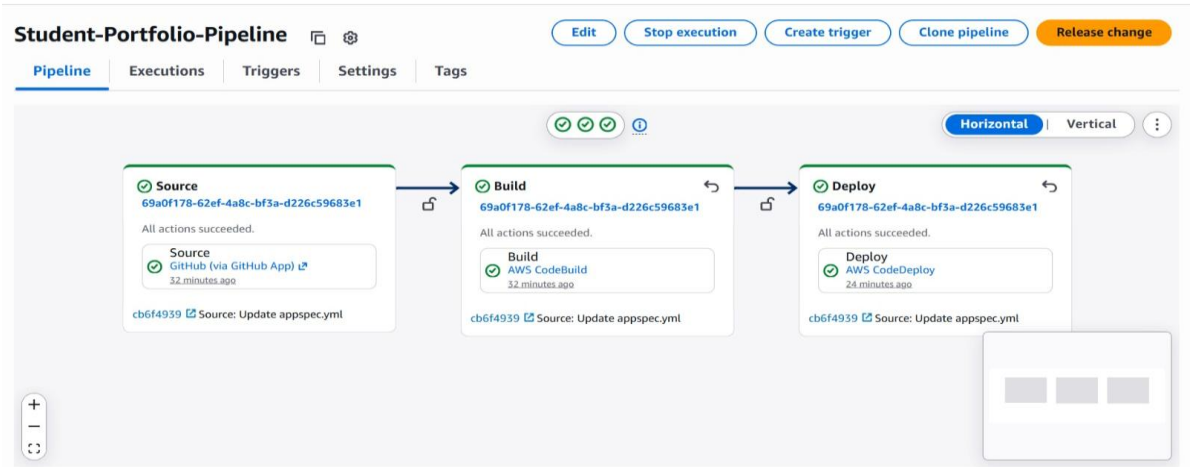


Figure 5: AWS CodePipeline interface showing successful Source, Build, and Deploy execution

6.5. Troubleshooting Solutions

During the implementation phase, we identified and resolved two primary issues:

Problem 1: CodeDeploy Agent Missing. The CodePipeline Deploy stage failed because the CodeDeploy agent was not active on the EC2 instances. Without this agent, the instance cannot receive deployment instructions from the CodeDeploy service.

Solution: We modified our EC2 User Data script to automatically install the agent during instance initialization. The following commands were executed to resolve the issue:

```
sudo yum update -y
sudo yum install ruby wget -y
cd /home/ec2-user
wget https://aws-codedeploy-eu-north-1.s3.eu-north-1.amazonaws.com/latest/install
chmod +x ./install
sudo ./install auto
sudo systemctl enable codedeploy-agent
sudo systemctl start codedeploy-agent
```

Problem 2: File Already Exists Conflict. Deployment failed during the file transfer step with an error stating that index.html already exists. CodeDeploy prevents file overwrites by default to protect existing files.

Solution: We created a scripts/deploy.sh file and registered it under the BeforeInstall hook. This script clears the /var/www/html directory before file transfer begins, ensuring clean deployments.

6. RESULT

Successful CI/CD Pipeline Execution

The implementation of the automated CI/CD pipeline using AWS was completed successfully, demonstrating an efficient and reliable software deployment process. The Student Portfolio website source code was stored in GitHub, and every code commit automatically triggered AWS CodePipeline through a webhook. This eliminated the need for manual deployment and enabled a fully automated workflow.

Build and Deployment Process

AWS CodeBuild successfully retrieved the source code, verified its structure, and generated a deployable build artifact, which was securely stored in an Amazon S3 bucket. AWS CodeDeploy then deployed the application to Amazon EC2 instances running the Apache HTTP Server. The entire deployment process was completed automatically without manual intervention, significantly reducing deployment time and minimizing configuration errors.

High Availability and Scalability

To ensure high availability and scalability, the application was hosted on Amazon EC2 instances within an Auto Scaling Group (ASG). The ASG maintained two running instances and was configured to scale up when required. An internet-facing Application Load Balancer (ALB) distributed incoming HTTP requests across the healthy EC2 instances, providing improved reliability and fault tolerance. The

deployed website remained accessible through the ALB DNS endpoint, and all instances successfully served the updated application.

Pipeline Validation and Performance

The pipeline was tested by making changes to the website source code and pushing the updates to the GitHub repository. Each code commit automatically triggered the complete CI/CD pipeline, and the latest version of the website was successfully deployed to all active EC2 instances in less than three minutes. This verified the correctness, speed, and efficiency of the automated deployment workflow.

Challenges Resolved

During implementation, two deployment issues were encountered and successfully resolved. The first issue was the absence of the CodeDeploy agent on the EC2 instances, which initially caused deployment failures. This was fixed by configuring an automated installation of the CodeDeploy agent through EC2 User Data during instance launch. The second issue occurred because existing website files in the Apache document root conflicted with the new deployment. This was resolved by creating a custom deployment script that removed the existing files before deploying the updated application.

Overall Outcome

The project successfully demonstrated that AWS CodePipeline, CodeBuild, and Code Deploy can automate the software release process efficiently. The implemented CI/CD pipeline provided faster deployments, improved application availability, reduced manual effort, and ensured a reliable, scalable, and fault-tolerant deployment environment.

7. CONCLUSION & FUTURE SCOPE

The successful implementation of this project demonstrates the effectiveness of AWS DevOps tools in automating software deployments. By linking GitHub, CodeBuild, CodeDeploy, and CodePipeline, we established a pipeline that automates deployment tasks. This reduces human error, provides version traceability, and establishes a foundation for zero-downtime updates.

The hosting environment uses an Auto Scaling Group and Application Load Balancer to ensure high availability. This setup scales compute resources based on user demand and handles instance failures without service interruption.

For future work, the pipeline and application infrastructure can be expanded with the following enhancements:

- Register a custom domain name using AWS Route 53 to map the Application Load Balancer DNS to a user-friendly URL.
- Configure SSL/TLS certificates via AWS Certificate Manager (ACM) to enable HTTPS on port 443.
- Set up automated unit testing in CodeBuild to validate HTML/CSS and run static code analysis before deployment.
- Integrate AWS Simple Notification Service (SNS) to email alerts to administrators in the event of pipeline failures.